United International University

Department of Computer Science and Engineering

Structured Programming Laboratory — Fall 2026

Topic: Operators and Expressions in C — Duration: 2.5 hours

Lab Sheet: Operator Related Problems

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU. Follow instructions carefully and complete all assignments within the designated slot.

1. Objectives

By the end of this lab, students will be able to:

- Apply arithmetic, assignment, increment, and decrement operators.
- Understand operator precedence, associativity, and type conversion.
- Use relational and logical operators to evaluate expressions.
- Apply the conditional operator to select a value.
- Use mathematical library functions to evaluate expressions.
- Determine the memory size of fundamental C data types using `sizeof`.

2. Introduction to Operators in C

Operators are symbols that perform operations on operands. C provides arithmetic operators (`+`, `-`, `*`, `/`, `%`), relational operators (`<`, `<=`, `>`, `>=`, `==`, `!=`), logical operators (`&&`, `||`, `!`), assignment operators (`=`, `+=`, `-=`, `*=`, `/=`), and increment/decrement operators (`++`, `-`).

Important notes

- Integer division discards the fractional part. For example, `5 / 10` evaluates to `0`.
- The remainder operator `%` works with integer operands only. For floating-point remainders, use `fmod()` from `math.h`.
- In C, a nonzero value is treated as true and zero is treated as false. Relational and logical expressions produce either `1` (true) or `0` (false).
- The postfix operators (`X++`, `X-`) use the current value before updating it. Prefix operators (`++X`, `-X`) update the value before it is used.
- Include `math.h` when using functions such as `sqrt()`, `sin()`, `cos()`, `ceil()`, `floor()`, and `fabs()`. Some toolchains require linking the math library (for example, `-lm`).

3. Arithmetic Operations (*)

Task: Write a program that takes two numbers, `X` and `Y`, as input and calculates their addition, subtraction, multiplication, division (integer quotient), and remainder. Use `double` values and `trunc()` and `fmod()` to calculate the quotient and remainder for the supplied floating-point examples.

arithmetic_operations.c

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double x, y;

    scanf("%lf %lf", &x, &y);

    printf("Addition: %.2f\n", x + y);
    printf("Subtraction: %.2f\n", x - y);
    printf("Multiplication: %.2f\n", x * y);

    if (y != 0) {
        printf("Quotient: %.0f\n", trunc(x / y));
        printf("Remainder: %.2f\n", fmod(x, y));
    } else {
        printf("Division is not possible.\n");
    }

    return 0;
}
```

Sample input

```
5 10
-5 10.5
```

Sample output for each run

```
Addition: 15.00
Subtraction: -5.00
Multiplication: 50.00
Quotient: 0
Remainder: 5.00
```

Note: The remainder of floating-point operands is calculated using `fmod()`, not the `%` operator.

4. Circumference of a Circle (*)

Task: Write a program that takes the radius `r` of a circle and calculates its circumference using the formula:

$$C = 2 \times \pi \times r$$

Use `3.14` as the value of π.

circle_circumference.c

```c
#include <stdio.h>

int main(void) {
    double r;
    const double PI = 3.14;

    scanf("%lf", &r);

    printf("Circumference: %.2f\n", 2 * PI * r);
    return 0;
}
```

Sample input

```
5
10.5
```

Sample output for each run

```
Circumference: 31.40
```

5. Evaluating an Algebraic Expression Without `math.h` (*)

Task: Take two numbers, `a` and `b`, as input and evaluate the following expression without using `math.h`. Use multiplication to calculate powers.

$$X = \frac{3.31a^2 + 2.01b^3}{7.16b^2 + 2.01a^3}$$

algebraic_expression.c

```c
#include <stdio.h>

int main(void) {
    double a, b, x;
    scanf("%lf %lf", &a, &b);

    double numerator = 3.31 * a * a + 2.01 * b * b * b;
    double denominator = 7.16 * b * b + 2.01 * a * a * a;

    if (denominator != 0) {
        x = numerator / denominator;
        printf("X = %.6f\n", x);
    } else {
        printf("The expression is undefined.\n");
    }

    return 0;
```

```
}
```

Sample input

```
5 10.5
```

Sample output

```
X = 2.315475
```

6. Increment and Decrement Operators (**)

Task: Write a program that demonstrates the prefix and postfix increment and decrement operators inside `printf()`. Reset `X` to the original input before each operation so that the four demonstrations are independent.

increment_decrement.c

```c
#include <stdio.h>

int main(void) {
    int x, original;
    scanf("%d", &original);

    x = original;
    printf("X++ : %d\n", x++);

    x = original;
    printf("++X : %d\n", ++x);

    x = original;
    printf("X-- : %d\n", x--);

    x = original;
    printf("--X : %d\n", --x);

    return 0;
}
```

Sample input

```
5
```

Sample output

```
X++ : 5
++X : 6
X– : 5
–X : 4
```

7. Incrementing and Decrementing Using Compound Assignment (*)

Task: Take two integers, `X` and `Y`, as input. Increment `X` by `Y` using the `+=` operator and decrement the original value of `X` by `Y` using the `-=` operator.

compound_addition_subtraction.c

```c
#include <stdio.h>

int main(void) {
    int x, y, original;
    scanf("%d %d", &x, &y);

    original = x;
    x += y;
    printf("Incremented Value: %d\n", x);

    x = original;
    x -= y;
    printf("Decremented Value: %d\n", x);

    return 0;
}
```

Sample input

```
5 10
```

Sample output

```
Incremented Value: 15
Decremented Value: -5
```

8. Multiplication and Division Using Compound Assignment (*)

Task: Take two integers, `X` and `Y`, as input. Use the `*=` operator to multiply `X` by `Y`, and use the `/=` operator to divide the original value of `X` by `Y`. Handle division by zero.

compound_multiplication_division.c

```c
#include <stdio.h>

int main(void) {
    int x, y, original;
    scanf("%d %d", &x, &y);

    original = x;
    x *= y;
    printf("Multiplication: %d\n", x);

    if (y != 0) {
        x = original;
```

```
        x /= y;
        printf("Division: %d\n", x);
    } else {
        printf("Division is not possible.\n");
    }

    return 0;
}
```

Sample input

```
56 10
```

Sample output

```
Multiplication: 560
Division: 5
```

9. Assignment and Type-Casting Conversions (**)

Task: Declare an integer and a floating-point variable. Read an integer and a floating-point value, then demonstrate (a) assigning the floating-point value to an integer, (b) assigning the integer to a floating-point variable, (c) explicitly casting the integer to `float`, and (d) explicitly casting the floating-point value to `int`.

type_conversion.c

```c
#include <stdio.h>

int main(void) {
    int i, converted_int;
    float f, converted_float;

    scanf("%d %f", &i, &f);

    converted_int = f;
    printf("Assignment: %.3f assigned to an int produces %d\n",
           f, converted_int);

    converted_float = i;
    printf("Assignment: %d assigned to a float produces %f\n",
           i, converted_float);

    printf("Type Casting: (float) %d produces %f\n",
           i, (float)i);

    printf("Type Casting: (int) %.3f produces %d\n",
           f, (int)f);

    return 0;
}
```

Sample input

```
-150 123.125
```

Sample output

```
Assignment: 123.125 assigned to an int produces 123
Assignment: -150 assigned to a float produces -150.000000
Type Casting: (float) -150 produces -150.000000
Type Casting: (int) 123.125 produces 123
```

Note: Conversion from floating point to integer discards the fractional part; it does not round to the nearest integer.

10. Finding the Maximum Using the Conditional Operator (**)

Task: Take two numbers, `x` and `y`, as input and print the larger value using the conditional operator (`?:`).

maximum_conditional.c

```c
#include <stdio.h>

int main(void) {
    int x, y, maximum;
    scanf("%d %d", &x, &y);

    maximum = (x > y) ? x : y;
    printf("Max: %d\n", maximum);

    return 0;
}
```

Sample input

```
20 100
```

Sample output

```
Max: 100
```

11. Operator Precedence and Expression Evaluation (*)

Task: Take `a`, `b`, and `c` as input and evaluate the following expressions. Pay attention to operator precedence and the use of parentheses.

$$X = a - b/3 + c \times 2 - 1$$
$$Y = a - (b/(3 + c) \times 2) - 1$$
$$Z = a - ((b/3) + c \times 2) - 1$$

Use integer variables and integer arithmetic.

precedence.c

```c
#include <stdio.h>

int main(void) {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    int x = a - b / 3 + c * 2 - 1;
    int y = a - (b / (3 + c) * 2) - 1;
    int z = a - ((b / 3) + c * 2) - 1;

    printf("X = %d\n", x);
    printf("Y = %d\n", y);
    printf("Z = %d\n", z);

    return 0;
}
```

Sample input

```
9 12 3
```

Sample output

```
X = 10
Y = 4
Z = -2
```

12. Relational and Logical Operators I (**)

Task: Take `a`, `b`, and `c` as input. Evaluate each of the following expressions and print the result as `1` (true) or `0` (false):

$$(a + b) \leq 80, \qquad !(a + c), \qquad a \neq 0$$

logical_operators_one.c

```c
#include <stdio.h>

int main(void) {
```

```c
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    printf("%d\n", (a + b) <= 80);
    printf("%d\n", !(a + c));
    printf("%d\n", a != 0);

    return 0;
}
```

Sample input

```
10 -10 0
```

Sample output

```
1
0
1
```

13. Relational and Logical Operators II (***)

Task: Take `a`, `b`, and `c` as input. Evaluate the following expressions and print each result as `1` or `0`. Carefully observe the precedence of `&&` and `||`.

$$(a + b) \leq 80 \;\&\&\; b \geq 0$$
$$(a - b) == 0 \;||\; c \neq 0$$
$$a \neq b \;||\; (b < a) \;\&\&\; c > 0$$

logical_operators_two.c

```c
#include <stdio.h>

int main(void) {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    printf("%d\n", (a + b) <= 80 && b >= 0);
    printf("%d\n", (a - b) == 0 || c != 0);
    printf("%d\n", a != b || (b < a) && c > 0);

    return 0;
}
```

Sample input

```
10 -10 0
```

Sample output

0
1
1

14. Roots of a Quadratic Equation (***)

Task: Take the coefficients `a`, `b`, and `c` of a quadratic equation

$$ax^2 + bx + c = 0$$

and calculate its roots using the quadratic formula:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

If the discriminant is negative, print `Imaginary`. Assume `a` is nonzero.

quadratic_roots.c

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double a, b, c, discriminant;
    scanf("%lf %lf %lf", &a, &b, &c);

    discriminant = b * b - 4 * a * c;

    if (discriminant < 0) {
        printf("Imaginary\n");
    } else {
        double root1 = (-b + sqrt(discriminant)) / (2 * a);
        double root2 = (-b - sqrt(discriminant)) / (2 * a);

        printf("%.2f    %.2f\n", root1, root2);
    }

    return 0;
}
```

Sample input

2 4 -16
1 2 3

Sample output for each run

2.00 -4.00

Imaginary

15. Trigonometric Expression (***)

Task: Take an angle `x` in degrees, where $1 \leq x \leq 180$, and evaluate the expression below. No input-range checking is required.

$$2\cos^2(x) - \sqrt{3}\sin(x) + \sin(x/2)$$

The trigonometric functions in `math.h` use radians. Convert the angle from degrees to radians before calling these functions.

trigonometric_expression.c

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double x, radian, result;
    const double PI = acos(-1.0);

    scanf("%lf", &x);
    radian = x * PI / 180.0;

    result = 2 * cos(radian) * cos(radian)
             - sqrt(3.0) * sin(radian)
             + sin(radian / 2.0);

    printf("%.10f\n", result);
    return 0;
}
```

Sample input

```
30
```

Sample output

```
0.8927936413
```

16. Rounding and Absolute Value (**)

Task: Take a floating-point number `X` as input and calculate:

- `A`: the ceiling of the absolute value of `X`.
- `B`: the floor of the absolute value of `X`.
- `C`: the absolute value of `X`.

Use `ceil()`, `floor()`, and `fabs()` from `math.h`.

rounding_absolute.c

```c
#include <stdio.h>
#include <math.h>
```

```c
int main(void) {
    double x, absolute_value;
    scanf("%lf", &x);

    absolute_value = fabs(x);

    printf("A = %.0f, B = %.0f, C = %.1f\n",
           ceil(absolute_value),
           floor(absolute_value),
           absolute_value);

    return 0;
}
```

Sample input

```
10.6
-77.9
```

Sample output for each run

```
A = 11, B = 10, C = 10.6
```

```
A = 78, B = 77, C = 77.9
```

17. Size of Fundamental Data Types (**)

Task: Write a program to find and print the size, in bytes, of `int`, `float`, `double`, and `char` on the system where the program is executed. Use the `sizeof` operator and the `%zu` format specifier.

data_type_sizes.c

```c
#include <stdio.h>

int main(void) {
    printf("Size of int in byte(s) = %zu\n", sizeof(int));
    printf("Size of float in byte(s) = %zu\n", sizeof(float));
    printf("Size of double in byte(s) = %zu\n", sizeof(double));
    printf("Size of char in byte(s) = %zu\n", sizeof(char));

    return 0;
}
```

Sample output

```
Size of int in byte(s) = 4
Size of float in byte(s) = 4
Size of double in byte(s) = 8
Size of char in byte(s) = 1
```

Note: The sizes shown above are common on many systems, but the sizes of most data types are implementation-dependent. The size of `char` is always 1 byte by definition.

Keep Coding!

Understanding operators and expressions is essential for writing correct and efficient C programs.